

บทที่ 4

SNMP

SNMP(SIMPLE NETWORK MANAGEMENT PROTOCOL)

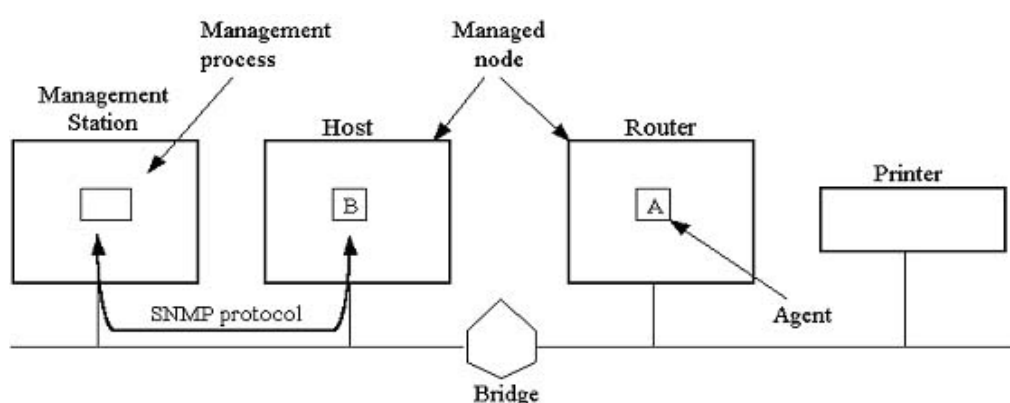
SNMP เป็นโปรโตคอลในระดับประยุกต์ (Application Layer) ที่กำหนดรูปแบบ และ กรรมวิธีการจัดการเครือข่าย โดยมีสถานีจัดการเครือข่ายส่วนกลางทำหน้าที่ดูแล ตรวจสอบ และควบคุมการทำงานของอุปกรณ์เครือข่าย

4.1 พื้นฐานการบริหารเครือข่าย

การบริหารเครือข่าย คือ การตรวจ ควบคุม และวางแผนการใช้ทรัพยากรระบบเพื่อให้เครือข่ายทำงานได้อย่างมีประสิทธิภาพ และสามารถตรวจหาจุดบกพร่องที่เกิดขึ้นเพื่อแก้ไขปัญหาได้อย่างรวดเร็ว

ในระบบเครือข่ายใดๆที่ เอสเอ็นเอ็มพี จัดการ จะต้องประกอบด้วย เอสเอ็นเอ็มพีโมเดล (SNMP Model) 4 ส่วนคือ

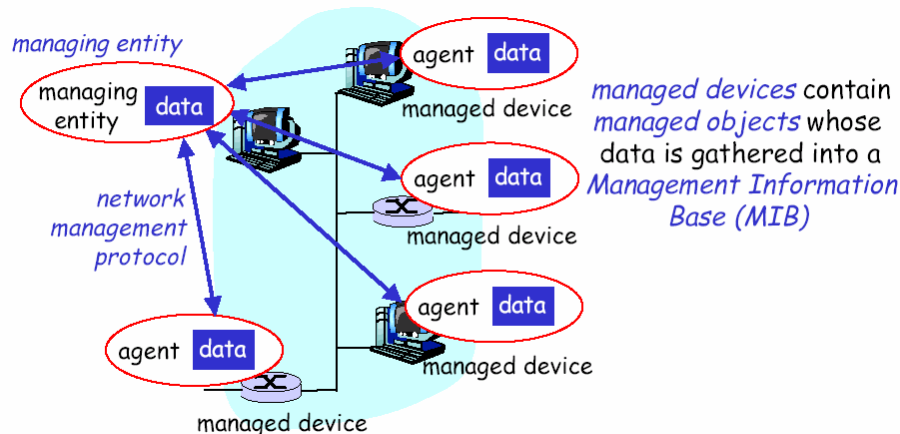
- 4.1.1. Managed nodes: อาจจะเป็น โฮสต์, เราท์เตอร์, ปริ้นเตอร์ หรืออุปกรณ์อื่นๆก็ได้ที่สามารถส่งข้อมูลสถานะของมันออกไปยังระบบได้ จะเป็นอุปกรณ์ ฮาร์ดแวร์ หรือ ซอฟต์แวร์ก็ได้ แต่ต้องมี เอสเอ็นเอ็มพีเอเจนต์ อยู่ในตัวด้วย
- 4.1.2. Management stations: คือ อุปกรณ์ใดๆที่มีฟังก์ชัน ให้ตรวจสอบและปรับเปลี่ยนการทำงานได้ ซึ่งทำหน้าที่ ตรวจตรา และ ควบคุม Managed nodes
- 4.1.3. Management information.
- 4.1.4. A management protocol.



รูปที่ 4-1 โมเดลส่วนประกอบการจัดการของเอสเอ็นเอ็มพี

4.2 เอสเอ็นเอ็มพีเอเจนต์

การจัดการเครือข่าย TCP/IP อาศัยรูปแบบการจัดการมาตรฐานตามข้อกำหนดของโปรโตคอล SNMP ซึ่งเป็นโปรโตคอลประยุกต์ที่กำหนดรูปแบบและกรรมวิธีการจัดการเครือข่าย โดยการทำงานของ agent จะนำข้อมูลจากส่วน ซอฟต์แวร์ หรือ ฮาร์ดแวร์ เมื่อ Management stations ร้องขอข้อมูล และปรับเปลี่ยนการทำงานของ ซอฟต์แวร์ หรือ ฮาร์ดแวร์ เมื่อ Management stations สั่งงาน โดยมีการแจ้งยืนยันสิทธิในรูปแบบในรหัสผ่านว่า Management stations มีอำนาจหน้าที่ในการร้องขอและปรับค่า



รูปที่ 4-2 องค์ประกอบในระบบจัดการเครือข่าย

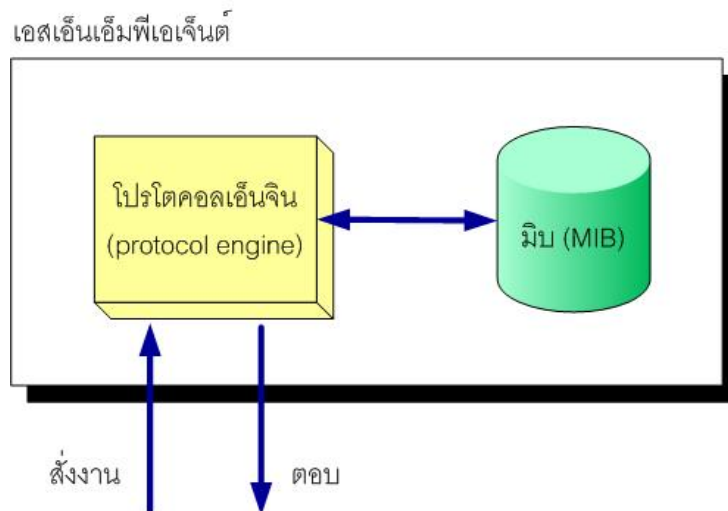
4.3 MIB (Management Information Base)

เอเจนต์จะประกอบด้วยส่วนสำคัญ 2 ส่วนคือ โปรโตคอลเอ็นจิน (Protocol engine) และฐานข้อมูลสารสนเทศการจัดการ (Management Information Base: MIB) โปรโตคอลเอ็นจินทำหน้าที่ประมวลคำสั่งที่มาจากNMS (Network Management Station) ซึ่งได้แก่ รับคำสั่ง ถอดรหัสคำสั่ง ทำงานตามคำสั่งและส่งผลตอบกลับ MIB เป็นส่วนที่เก็บตัวแปรและค่ากำหนดการทำงานประจำอุปกรณ์

ภายใน MIB ประกอบด้วยตัวแปรจำนวนมากที่เรียกโดยทั่วไปว่า อ็อบเจกต์จัดการ (managed object) อ็อบเจกต์ในความหมายนี้เป็นชื่อที่ใช้เรียกตัวแปรและลักษณะเฉพาะของตัวแปรในMIBโดยไม่เกี่ยวข้องกับเชิงวัตถุพิสัย (object oriented) แต่อย่างใด โดยจะพิจารณาอ็อบเจกต์ใน SNMP มีลักษณะเช่นเดียวกับเรคอร์ดในฐานข้อมูล

แต่ละอ็อบเจกต์ จะมีชื่อเรียกเฉพาะเรียกว่า อ็อบเจกต์ไอดีไฟเอร์ (Object Identifier) หรือเรียกโดยย่อว่า ไอดีไฟเอร์ (Identifier) เพื่อใช้อ้างอิงถึงอ็อบเจกต์นั้น

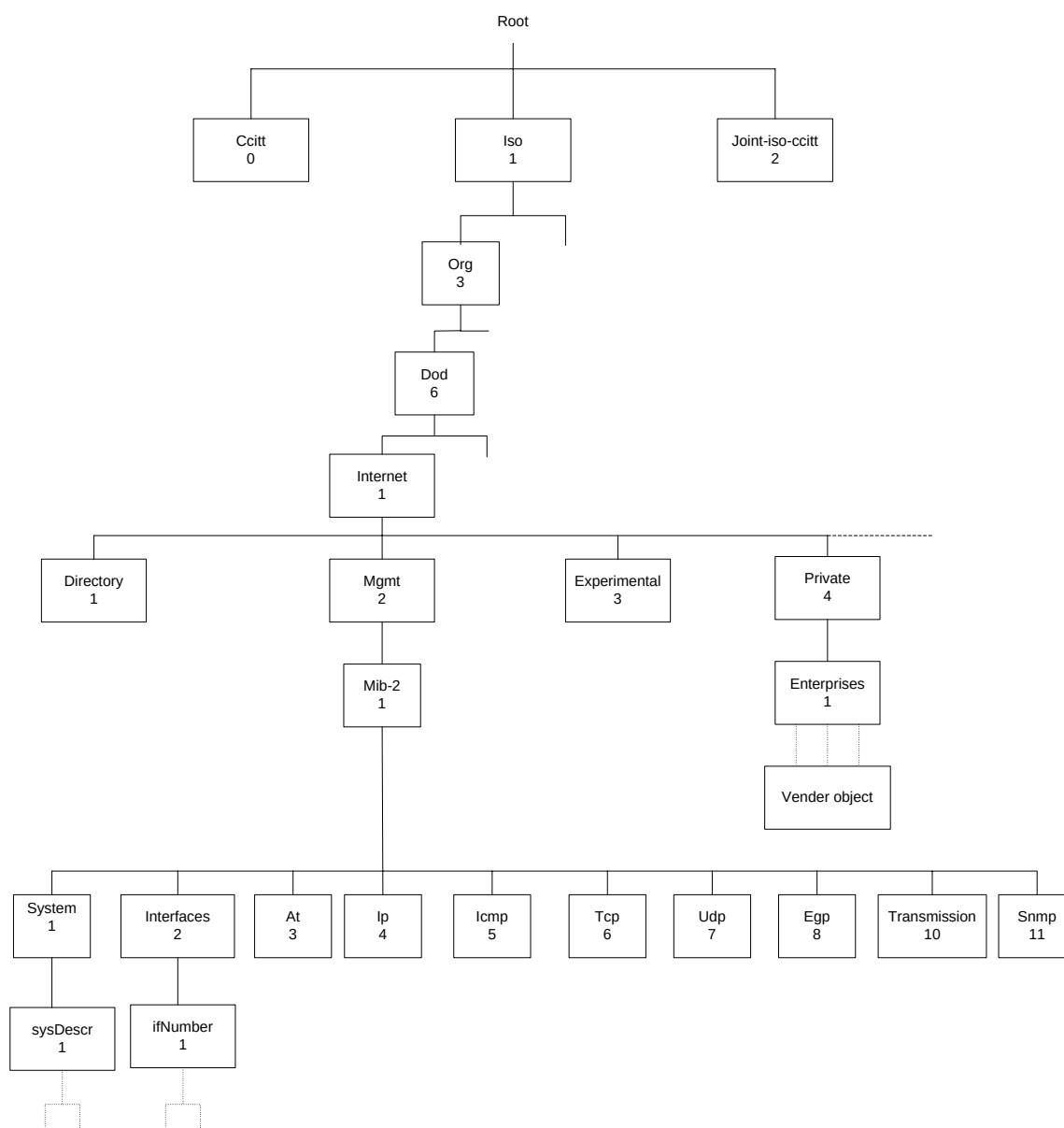
อ็อบเจกต์ทุกตัวมีนิยามที่กำหนด ชื่อ แบบข้อมูล สิทธิการเข้าถึง คำอธิบายลักษณะ และค่าข้อมูล การนิยามอ็อบเจกต์มีกฎเกณฑ์ตามข้อกำหนด โครงสร้างฐานข้อมูลสารสนเทศ (Structure of Management Information: SMI) [RFC 1155]



รูปที่ 4-3 โครงสร้างของเอเจนต์

4.3.1 โครงสร้าง MIB

ข้อมูลประจำอุปกรณ์เครือข่ายชั้นหนึ่งๆมีได้อย่างหลากหลาย อีกทั้งอุปกรณ์ต่างประเภทกันย่อมมีข้อมูลประจำอุปกรณ์แตกต่างกัน ดังนั้นการสอบถาม (อ่าน) หรือเปลี่ยนค่า (เขียน) ฐานข้อมูลจึงต้องมีรูปแบบมาตรฐานให้กับอุปกรณ์ทุกประเภท โครงสร้างต้นไม้แบบลำดับชั้นเป็นโครงสร้างที่เหมาะสมสำหรับใช้เป็นฐานข้อมูลเพื่อจัดเก็บตัวแปรเหล่านี้



รูปที่ 4-4 อ็อบเจกต์ไอดีเอ็นดีไฟเบอร์ในโครงสร้างฐานข้อมูลสารสนเทศการจัดการ

ดังรูปที่ 4-4 แสดงข้อมูลหรืออ็อบเจกต์ของเอสเอ็นเอ็มพีในโครงสร้างต้นไม้ซึ่งนิยมเรียกว่า มิบทรี (MIB Tree) แต่ละโหนดซึ่งแทนอ็อบเจกต์หนึ่งๆมีชื่อพร้อมทั้งตัวเลขฐานสิบกำกับประจำโหนดเพื่อใช้อ้างอิง ยกเว้นรากซึ่งไม่มีชื่อกำกับ

ลำดับชั้นแรกจะมีโหนดหลัก 3 โหนดซึ่งกำหนดกลุ่มองค์กร 3 กลุ่มคือ ITU-T(0),ISO(1) และ Joint-ISO-ITU-T(2) ภายใต้โหนด ISO มีโหนดลำดับที่ 3 คือ org(3) กำหนดองค์กรนานาชาติ และส่วนหนึ่งขององค์กรนี้คือ dod (6) หรือ Department of Defense และมี โหนด internet(1) เพื่อกำหนดกลุ่มการจัดการเครือข่ายใน internet

เมื่อต้องการอ้างอิงถึงโหนดใดในโครงสร้าง ให้เขียนหมายเลขจากรากไปตามเส้นทางถึงโหนดนั้นและคั่นด้วยจุด ลำดับตัวเลขนี้เรียกว่า อ็อบเจกต์ไอดีเอ็นดีไฟเบอร์(Object Identifier) หรือ โอไอดี (OID)

ตัวอย่างเช่น 1.3.6.1.2.1.1 เป็นอ็อบเจกต์ไอดีเน็ตเวิร์กโดยมีชื่อที่สมนัยกันคือ iso.org.dod.internet.mgmt.mib-2.system โหนดที่อยู่ภายใต้ 1.3.6.1.2.1 หรือในกลุ่ม mib-2 เป็นโหนดสำหรับใช้งานเอสเอ็นเอ็มพี แต่ละโหนดจะมีโหนดย่อยเพื่ออ้างอิงถึงตัวแปรเช่น 1.3.6.1.2.1.1.1 คือตัวแปร sysDescr (System Description) ซึ่งเก็บคำอธิบายเกี่ยวกับอุปกรณ์นั้น

4.3.2 กลุ่มในมิบ

มิบภายใต้ internet มีกลุ่มย่อยทั้งหมด 6 กลุ่มคือ

- directory(1) สงวนไว้สำหรับใช้งานในอนาคต
- mgmt(2) กลุ่มมิบที่ใช้ในการจัดการภายใต้เอสเอ็นเอ็มพีรุ่น 1
- experimental(3) ใช้สำหรับการทดลอง
- private(4) สำหรับผู้ผลิตกำหนดตัวแปรเฉพาะอุปกรณ์
- security(5) ใช้ในระบบรักษาความปลอดภัย
- SNMPv2(6) ใช้ในเอสเอ็นเอ็มพีรุ่น 2

ภายใต้กลุ่ม mib-2 บรรจุกลุ่มย่อยที่ใช้ในเอสเอ็นเอ็มพีซึ่งประกอบด้วย interface, at, ip และอื่นๆ ความหมายของแต่ละกลุ่มอธิบายไว้ในตารางที่ 4-1 แต่ละกลุ่มประกอบด้วยตัวแปรซึ่งมีแบบต่างๆกันไป

ลำดับ	ชื่อ	ความหมาย
1	system	ข้อมูลระบบ
2	interface	ข้อมูลอินเทอร์เฟซที่ใช้เชื่อมต่อ
3	at	ข้อมูลการแปลงแอดเดรส
4	ip	ข้อมูลไอพี
5	icmp	ข้อมูลไอซีเอ็มพี
6	tcp	ข้อมูลทีซีพี
7	udp	ข้อมูลยูดีพี
8	egp	ข้อมูลโปรโตคอลเกตเวย์ภายนอก
10	transmission	ข้อมูลสายสื่อสาร
11	SNMP	ข้อมูลเอสเอ็นเอ็มพี

ตารางที่ 4-1 กลุ่มย่อยภายใต้ mgmt

4.3.3 ชนิดของตัวแปรมิม

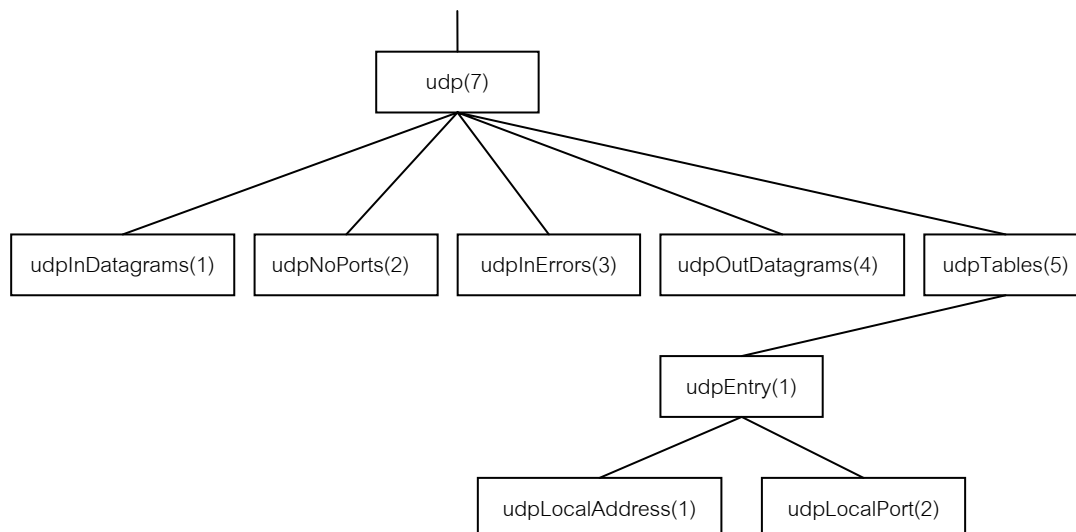
แต่ละตัวแปรในเอสเอ็นเอ็มพีมีข้อมูลประจำ แบบข้อมูลที่ให้อยู่ในเอสเอ็นเอ็มพีมีดังนี้

- Integer: จำนวนเต็มเช่นหมายเลขพอร์ตของโปรโตคอลที่ซีพีหรือยูดีพี มีค่าได้ตั้งแต่ 0 ถึง 65535
- OctedString: สายอักขระขนาดตั้งแต่ 0 อ็อกเทต แต่ละอ็อกเทตมีค่าตั้งแต่ 0 ถึง 255 ตัวอย่างแบบข้อมูลสายอักขระได้แก่รหัสผ่าน
- DisplayString: สายอักขระตั้งแต่ 0 อ็อกเทตแต่ละอ็อกเทตต้องเป็นรหัสแอสกีเอ็นวีที ข้อมูลประเภทนี้มีความยาวตั้งแต่ 0 ถึง 255 ตัวอักษร
- Null: ใช้บอกว่าตัวแปรนั้นไม่มีค่าข้อมูลใดอยู่ เช่นเมื่อสอบถามข้อมูลด้วยคำสั่ง get หรือ get-next-request จะกำหนดแบบข้อมูลตัวแปรเท่ากับ null
- ObjectIdentifier: ชื่อตัวแปรในรูปแบบของการอ้างถึงแบบตัวเลขตามโครงสร้างมิม
- IpAddress: สายอักขระ 4 อ็อกเทต แต่ละอ็อกเทตแทนไอพีแอดเดรสแต่ละตำแหน่ง
- PhysicalAddress: สายอักขระกำหนดฮาร์ดแวร์แอดเดรสเช่นอีเทอร์เน็ตแอดเดรสใช้สายอักขระ 6 อ็อกเทต
- Counter: เลขจำนวนเต็มไม่คิดเครื่องหมาย มีค่าตั้งแต่ 0 ถึง $2^{31} - 1$ (4,294,967,295) การใช้ข้อมูล counter เป็นแบบเพิ่มค่าขึ้นอย่างเดียว เมื่อเพิ่มถึงค่ามากที่สุดจะกลับเป็น 0 ใหม่
- Gauge: เลขจำนวนเต็มไม่คิดเครื่องหมาย มีค่าตั้งแต่ 0 ถึง $2^{31} - 1$ โดยสามารถเพิ่มหรือลดค่าได้ แต่เมื่อเพิ่มไปสูงสุดแล้วจะคงค่าไว้จนกว่าจะถูกปรับค่ากลับมาเป็น 0 อีกครั้ง ตัวอย่างตัวแปรที่ใช้ค่านี้นั้น จำนวนการเชื่อมโยงที่ซีพีที่อนุญาตให้มีได้
- TimeTicks: เลขจำนวนเต็มใช้นับเวลาให้หน่วยเศษหนึ่งส่วนร้อยของวินาที เช่น เวลานั้นตั้งที่ระบบเริ่มทำงาน
- Sequence: โครงสร้างแบบเรคคอร์ด หรือคล้ายกับแบบข้อมูล struct ในภาษาซี
- Sequence of: โครงสร้างแบบตารางหรือมองในรูปของอาร์เรย์ เช่นตารางเลือกเส้นทางของไอพี

4.3.4 ตัวอย่างแบบข้อมูลอาร์เรย์

แบบข้อมูล sequence of ใช้กำหนดข้อมูลแบบเวกเตอร์ซึ่งสมาชิกทั้งหมดภายในมีข้อมูลแบบเดียวกัน หากสมาชิกมีแบบข้อมูลเบื้องต้น เช่น แบบจำนวนเต็มก็จะได้เวกเตอร์แบบมิติเดียว หรือหากสมาชิกมีข้อมูลแบบ sequence ก็จะได้อาร์เรย์ 2 มิติ

ตัวอย่างของตัวแปรโครงสร้างอาร์เรย์ในมิมมีอยู่หลายตัวแปร แต่จะยกตัวอย่างเฉพาะตัวแปรที่มีขนาดเล็กเพื่อที่จะทำความเข้าใจได้ง่ายได้แก่ udpTable ซึ่งตั้งกัอยู่ภายใต้กลุ่ม udp ตามโครงสร้างข้อมูลดังรูปที่ 4-5



รูปที่ 4-5 กลุ่ม udp

udpTable มีแบบข้อมูล sequence of และภายใต้ udpTable มี udpEntry ซึ่งมีแบบข้อมูล sequence โดยประกอบด้วย udpLocalAddress และ udpLocalPort

udpLocalAddress มีแบบข้อมูล IPAddress ใช้กำหนดไอพีแอดเดรสที่รอให้บริการส่วนของ udpLocalPort กำหนดหมายเลขพอร์ต ดังนั้น udpTable จึงมีโครงสร้างเป็นอาร์เรย์ 2 มิติหรือเขียนด้วยตาราง

ดังรูปที่ 4-6

	udpLocalAddress	udpLocalPort
udpEntry	(IpAddress)	(Integer)
udpEntry	...	...
udpEntry	...	...
udpEntry	...	...
udpEntry	...	...

รูปที่ 4-6 udpTable ในรูปอาร์เรย์สองมิติ (ตาราง)

4.4 การแทนข้อมูลด้วย ASN.1

ไอเอสโอและซีซีไอทีที่กำหนดวิธีการนิยามชนิดของตัวแปร โดยใช้ไวยากรณ์ ASN.1 (Abstract Syntax Notation One) ซึ่งเป็นเป็นเสมือนภาษาอธิบายแบบข้อมูลที่ไม่ขึ้นกับฮาร์ดแวร์ต้นกำเนิดของ

ASN.1 นำมาใช้ในมาตรฐานโพรโทคอลของไอเอสโอ แต่สำหรับเอสเอ็นเอ็มพีจะใช้เพียงไวยากรณ์เพียงบางส่วนของ ASN.1 เท่านั้น

4.4.1 Module Definition

การใช้ ASN.1 ช่วยให้ผู้นามาตราฐานไปใช้เข้าใจถึงสิ่งที่ผู้สร้างมาตรฐานกำหนดไว้โดยไม่เกิดความกำกวมในเรื่องของความหมายและการแทนข้อมูล เช่นการกำหนดตัวแปรชนิด integer จะต้องกำหนดให้แน่นอนว่ามีค่าอยู่ในช่วงใด เพราะว่าคอมพิวเตอร์ต่างชนิดกันอาจมีความแตกต่างกันในการแทนข้อมูลและช่วงของตัวแปรที่แตกต่างกัน มีบ๊อบเจ็กในเอสเอ็นเอ็มพีมีรูปแบบที่กำหนดด้วย ASN.1 ตัวอย่างต่อไปนี้เป็นนิยามของบ๊อบเจ็ก sysContact

```
Syscontact      OBJECT-TYPE
                SYNTAX  DisplayString (size (0..255))
                ACCESS   read-write
                STATUS    mandatory
                DESCRIPTION
                    "The textual identification of the contact person for this managed node,
                    together with information on how to contact this person."
                ::= (system 4)
```

นิยามข้างต้นมีความหมายดังนี้

- SYNTAX : กำหนดแบบข้อมูลของตัวแปรเช่นจำนวนเต็มหรืออักขระ ในที่นี้คือ DisplayString
- ACCESS : แบบการใช้ซึ่งอาจเป็น read-only(อ่านอย่างเดียว), read-write(อ่านเขียนได้), write-only(เขียนอย่างเดียว) หรือ not-accessible (ห้ามเข้าถึง) เป็นต้น
- STATUS : กำหนดสถานะของตัวแปรว่าจำเป็นต้องมีตัวแปรนี้หรือไม่ ค่าที่เป็นไปได้ เช่น mandatory(จำเป็น), optional(ออฟชั่นซึ่งมีหรือไม่ก็ได้), deprecate (จำเป็นต้องมีแต่อาจยกเลิกในรุ่นถัดไป), obsoleted (ไม่จำเป็นเนื่องจากยกเลิกไม่ใช้แล้ว)
- DESCRIPTION : ข้อความอธิบายตัวแปร
- บรรทัดสุดท้ายของนิยามกำหนดว่าตัวแปร sysContact จะเชื่อมกับโครงสร้างต้นไม้ต่อจากโหนด system และมีค่าเท่ากับ 4 ซึ่งแสดงถึงไอดีเน็ตเวิร์กประจำ sysContact คือ iso.org.dod.internet.mgmt.mib-2.system.4 หรือ 1.3.6.1.2.1.1.4

ASN.1 คือภาษาที่สามารถใช้ในการกำหนดโครงสร้างข้อมูล โดยโครงสร้างที่ถูกกำหนดนี้จะอยู่ในรูปแบบที่ชื่อว่าโมดูล โดยชื่อของโมดูลสามารถที่จะถูกใช้อ้างอิงถึงโครงสร้างได้ ตัวอย่างเช่น ชื่อโมดูล

สามารถที่จะถูกใช้เสมือน ชื่อ abstract syntax ซึ่ง application สามารถที่จะส่งชื่อไปเพื่อกำหนด abstract syntax ของ APDUs ที่ application ต้องการจะเปลี่ยน โดยจะมีรูปแบบพื้นฐานของโมดูลเป็นดังนี้

```
<modulereference> DEFINITIONS ::=
    BEGIN
        EXPORTS
        IMPORTS
        Assignmentlist
    End
```

เมื่อ Modulereference :: ชื่อของโมดูลนั้นๆ
 EXPORTS :: เป็นตัวชี้ว่าโมดูลนี้อาจจะถูกอิมพอร์ตจากโมดูลอื่น
 IMPORTS :: เป็นตัวบอกว่ารูปแบบและค่าจากโมดูลอื่น ได้ถูกอิมพอร์ตมา
 โมดูลนี้
 Assignment :: จะประกอบด้วย ชนิดของ assignment ค่าของ assignment และ
 การกำหนดค่าของมาโคร

4.4.2 Macro Definitions

เป็นการอนุญาตให้ผู้ใช้งานสามารถที่จำขยาย syntax ของ ASN.1 โดยการกำหนดรูปแบบใหม่และค่าของรูปแบบนั้นๆ โดยจำมีระดับของกำหนดคือ

Macro definition: เป็นการกำหนดกฎของ macro instance โดยจะกำหนด syntax ของเซตของรูปแบบที่เกี่ยวข้อง

Macro instance: ถูกสร้างจาก การกำหนด macro instance

Macro instance value: การกำหนดค่าของ macro instance

การกำหนดมาโครจะมีรูปแบบดังนี้

```
<macroname> MACRO ::=
    BEGIN
        TYPE NOTATION ::= <new-type-syntax>
        VALUE NOTATION ::= <new-value-syntax>
        <supporting-productions>
    END
```

ตัวอย่างการกำหนดมาโคร

OBJECT-TYPE MACRO ::=

BEGIN

TYPE NOTATION ::= “Syntax” type (TYPE ObjectSyntax)

“ACCESS” Access

“STATUS” Status

VALUE NOTATION ::= value (VALUE ObjectName)

Access ::= “read-only”|“read-write”|“write-only”|“not-accessible”

Status ::= “mandatory”|“optional”|“obsolete”

END

4.4.3 Defining Tables

การกำหนดฐานข้อมูลของอ็อบเจกต์นั้นจะมีการเกี่ยวกับการใช้ sequence และ sequence-of โดยทางที่ดีที่สุดที่จะอธิบายนั้นคงจะเป็นการยกตัวอย่างให้ดู โดยที่เราจะพิจารณาที่ tcpConnTable ที่มีอ็อบเจกต์ไอเอ็นดีไฟเบอร์เป็น 1.3.6.1.2.1.6.13 ตาราง TCP connection จะประกอบด้วย ข้อมูลเกี่ยวกับรูปแบบการติดต่อของ TCP ไอพีแอดเดรสและพอร์ต ของเครื่องที่ติดต่อ จาก code ตัวอย่างด้านล่างสามารถบอกเราได้ว่า มีการประกาศตารางซึ่งประกอบด้วย TcpConnEntry ซึ่ง TcpConnEntry จะมี tcpConnState(รูปแบบการติดต่อของ TCP), tcpConnLocalAddress (ไอพีแอดเดรสของเครื่องที่ติดต่อ), tcpConnLocalPort(พอร์ตของเครื่องที่ติดต่อ), tcpConnRemAddress(ไอพีแอดเดรสของเครื่องที่ติดต่ออีกเครื่องหนึ่ง), tcpConnRemPort(พอร์ตของเครื่องที่ติดต่ออีกเครื่องหนึ่ง)

tcpConnTable OBJECT-TYPE

SYNTAX SEQUENCE OF TcpConnEntry

ACCESS not-accessible

STATUS mandatory

DESCRIPTION

"A table containing TCP connection-specific
information."

::= { tcp 13 }

tcpConnEntry OBJECT-TYPE

SYNTAX TcpConnEntry

ACCESS not-accessible

STATUS mandatory

DESCRIPTION

"Information about a particular current TCP connection. An object of this type is transient, in that it ceases to exist when (or soon after) the connection makes the transition to the CLOSED state."

INDEX { tcpConnLocalAddress,
tcpConnLocalPort,
tcpConnRemAddress,
tcpConnRemPort }

::= { tcpConnTable 1 }

TcpConnEntry ::=

SEQUENCE {tcpConnState INTEGER,
tcpConnLocalAddress IpAddress,
tcpConnLocalPort INTEGER (0..65535),
tcpConnRemAddress IpAddress,
tcpConnRemPort INTEGER (0..65535)}

tcpConnState OBJECT-TYPE

SYNTAX INTEGER {closed(1),
listen(2),
synSent(3),
synReceived(4),
established(5),
finWait1(6),
finWait2(7),
closeWait(8),
lastAck(9),
closing(10),
timeWait(11),
deleteTCB(12)}

ACCESS read-write

STATUS mandatory

DESCRIPTION

"The state of this TCP connection.

The only value which may be set by a management station is deleteTCB(12). Accordingly, it is

appropriate for an agent to return a 'badValue' response if a management station attempts to set this object to any other value.

If a management station sets this object to the value deleteTCB(12), then this has the effect of deleting the TCB (as defined in RFC 793) of the corresponding connection on the managed node, resulting in immediate termination of the connection.

As an implementation-specific option, a RST segment may be sent from the managed node to the other TCP endpoint (note however that RST segments are not sent reliably)."

::= { tcpConnEntry 1 }

tcpConnLocalAddress OBJECT-TYPE

SYNTAX IPAddress

ACCESS read-only

STATUS mandatory

DESCRIPTION

"The local IP address for this TCP connection. In the case of a connection in the listen state which is willing to accept connections for any IP interface associated with the node, the value 0.0.0.0 is used."

::= { tcpConnEntry 2 }

tcpConnLocalPort OBJECT-TYPE

SYNTAX INTEGER (0..65535)

ACCESS read-only

STATUS mandatory

DESCRIPTION

"The local port number for this TCP connection."

::= { tcpConnEntry 3 }

tcpConnRemAddress OBJECT-TYPE

SYNTAX IPAddress

ACCESS read-only

STATUS mandatory

DESCRIPTION

"The remote IP address for this TCP connection."

::= { tcpConnEntry 4 }

tcpConnRemPort OBJECT-TYPE

SYNTAX INTEGER (0..65535)

ACCESS read-only

STATUS mandatory

DESCRIPTION

"The remote port number for this TCP connection."

::= { tcpConnEntry 5 }

tcpConnTable (1.3.6.1.2.1.6.13)

tcpConnState (1.3.6.1.2.1.6.13.1.1)	tcpConnLocalAddress (1.3.6.1.2.1.6.13.1.2)	tcpConnLocalPort (1.3.6.1.2.1.6.13.1.3)	tcpConnRemAddress (1.3.6.1.2.1.6.13.1.4)	tcpConnRemPort (1.3.6.1.2.1.6.13.1.5)
--	---	--	---	--

5	10.0.0.99	12	9.1.2.3	15
2	0.0.0.0	99	0.0.0.0	0
3	10.0.0.99	14	89.1.1.42	84

↑
INDEX

↑
INDEX

↑
INDEX

↑
INDEX

tcpConnEntry (1.3.6.1.2.1.6.13.1)

tcpConnEntry (1.3.6.1.2.1.6.13.1)

tcpConnEntry (1.3.6.1.2.1.6.13.1)

รูปที่ 4-7 ตาราง tcpConnTable

4.5 Communities and Community Names

การบริหารเครือข่ายจะถือได้ว่าเป็นการทำงานในลักษณะระบบกระจาย (Distributed application) รูปแบบหนึ่ง ซึ่งจะเห็นได้ว่าความสัมพันธ์ระหว่างสถานีจัดการเครือข่าย (Network Management Station : NMS) กับเอเจนต์ (Agent) จะเป็นในรูปแบบ many-to-many คือ สถานีจัดการเครือข่ายจะบริหารจัดการเอเจนต์ได้หลายเครื่อง และในขณะเดียวกันเอเจนต์ก็สามารถถูกบริหารควบคุมจากสถานีจัดการเครือข่ายหลายเครื่องเช่นกัน

จากความสัมพันธ์ดังกล่าวจึงจำเป็นต้องมีมาตรการควบคุมความปลอดภัยในการใช้งานฐานข้อมูลสารสนเทศการจัดการ (Management Information Base : MIB) ของตนเองโดยจะมีมุมมองทางด้านความปลอดภัย 3 ประการได้แก่

- การพิสูจน์ตัวตน (Authentication service) จะเป็นการจำกัดให้เฉพาะสถานีจัดการเครือข่ายที่จะเข้ามาบริการควบคุม
- นโยบายการเข้าถึง (Access policy) จะมีการกำหนดระดับการอนุญาตการเข้าถึงฐานข้อมูลสารสนเทศการจัดการให้แต่ละสถานีจัดการเครือข่ายไม่เท่ากันในแต่ละเครื่อง
- การให้บริการ Proxy (Proxy service) เอเจนต์อาจทำหน้าที่เป็น Proxy ให้กับเอเจนต์เครื่องอื่นซึ่งจะรวมถึงการพิสูจน์ตัวตนและนโยบายการเข้าถึงของเอเจนต์ตัวอื่นที่อยู่ในระบบ Proxy

เอสเอ็นเอ็มพี (SNMP) ได้มีการกำหนดการทำงานเพื่อสนับสนุนมุมมองทางด้านความปลอดภัยดังกล่าวในรูปแบบของ SNMP Community โดยการทำงานคือ เอเจนต์แต่ละเครื่องจะมีการสร้าง Community name เพื่อกำหนดให้กับสถานีจัดการเครือข่าย โดเมนหนึ่ง Community name จะสามารถมีสถานีจัดการเครือข่ายมากกว่าหนึ่งตัว

เนื่องจาก Community name จะถูกกำหนดในแต่ละเอเจนต์จึงอาจเป็นไปได้ว่ามีการตั้งชื่อ Community name ซ้ำกันในแต่ละเอเจนต์ แต่สถานีจัดการเครือข่ายจะสามารถแยกความแตกต่างของ Community ที่มีชื่อซ้ำกันเหล่านี้เองได้ถ้าอยู่ในคนละเอเจนต์กัน ดังนั้นจึงจำเป็นต้องจำเป็นอย่างยิ่งว่าสถานีจัดการเครือข่ายจะต้องเก็บข้อมูลของ Community name และข้อมูลที่เกี่ยวข้องของแต่ละเอเจนต์เพื่อใช้ในการบริหารควบคุม

4.5.1 การพิสูจน์ตัวตน (Authentication service)

การพิสูจน์ตัวตนมีไว้เพื่อให้แน่ใจการติดต่อนั้นเป็นของแท้ ในกรณีของเอสเอ็นเอ็มพีการพิสูจน์ตัวตนจะมีไว้เพื่อให้แน่ใจว่า message ที่ได้รับมานั้นเป็นข้อความที่แท้จริง โดยในทุกๆ message ของเอสเอ็นเอ็มพีจะมีการระบุ Community name ซึ่งจะมีหน้าที่เหมือนกับรหัสผ่าน (Password) ในการพิสูจน์ตัวตน นอกจากนั้นยังอาจจะมีการเข้ารหัส (Encryption) เพื่อเพิ่มความปลอดภัยในการพิสูจน์ตัวตนมากยิ่งขึ้น

4.5.2 นโยบายการเข้าถึง (Access policy)

การควบคุมการเข้าถึงในเอสเอ็นเอ็มพีจะประกอบด้วย 2 องค์ประกอบหลักที่เกี่ยวข้องคือ

- SNMP MIB view: คือกลุ่มของอ็อบเจกต์ในฐานข้อมูลสารสนเทศการจัดการที่ตั้งขึ้นโดยในแต่ละกลุ่มอาจจะประกอบด้วยหลาย Sub tree ในฐานข้อมูลสารสนเทศการจัดการได้
- SNMP access mode: คือรูปแบบของการเข้าถึงได้แก่ READ-ONLY และ READ-WRITE

ในเอเจนต์จะมีการกำหนด Access mode ให้แต่ละ MIB view ซึ่ง Access mode จะมีผลกับทุกๆ Object ที่อยู่ในกลุ่มของ MIB view โดยทั้ง Access mode และ MIB view จะถูกรวบรวมกันว่า SNMP community profile ซึ่งจะถูกกำหนดในแต่ละ Community

ใน Access mode ของเอสเอ็นเอ็มพีจะมีการประนีประนอมต่อรองระดับการเข้าถึงกับระดับของการเข้าถึงของฐานข้อมูลสารสนเทศการจัดการ (MIB Access Category) ดังตารางที่ 2 (Access mode ของเอสเอ็นเอ็มพีกับระดับของการเข้าถึงของฐานข้อมูลสารสนเทศการจัดการเป็นคนละส่วนกัน) และจะเรียกรวม SNMP community และ SNMP community profile ว่า SNMP access policy

MIB Access Category	SNMP Access Mode	
	READ-ONLY	READ-WRITE
read-only	สามารถใช้คำสั่ง get และ trap ได้	
read-write	สามารถใช้คำสั่ง get และ trap ได้	สามารถใช้คำสั่ง get, set และ trap ได้
write-only	สามารถใช้คำสั่ง get และ trap แต่ต้องเป็นค่าที่เป็น implementation-specific	สามารถใช้คำสั่ง get, set และ trap ได้แต่ต้องเป็นค่าที่เป็น implementation-specific สำหรับคำสั่ง get และ trap
not accessible	ไม่สามารถเข้าถึงได้	

ตารางที่ 4-2 ความสัมพันธ์ระหว่าง MIB Access Category และ SNMP Access Mode

4.5.3 การให้บริการ Proxy (Proxy service)

แนวคิดของ Community จะสามารถสนับสนุนการทำ Proxy ได้ โดยเอเจนต์จะสามารถทำหน้าที่เป็นตัวแทนในการติดต่อกับสถานีจัดการเครือข่ายให้กับเอเจนต์หรืออุปกรณ์อื่นได้ ในกรณีที่อุปกรณ์เครื่องนั้นไม่สนับสนุน TCP/IP และเอสเอ็นเอ็มพีหรือในกรณีที่ต้องการลดปริมาณการติดต่อระหว่างอุปกรณ์นั้นกับสถานีจัดการเครือข่าย โดยเอเจนต์ที่เป็น Proxy จะสนับสนุน SNMP access policy ของเอเจนต์ที่อยู่ในระบบ Proxy ด้วย

4.6 การอ้างอิงถึงค่าในอ็อบเจกต์ (Instance Identification)

ในการอ้างอิงถึงค่าของอ็อบเจกต์ในฐานข้อมูลสารสนเทศการจัดการของเอสเอ็นเอ็มพีนั้นจะอาศัยการอ้างอิงของอินสแตนซ์ไอดีไฟเดอร์ (Instance Identifier)

สำหรับการอ้างอิงถึงค่าของอ็อบเจกต์แบบปกติ (Simple Object Value) โดยทั่วไปจะใช้อินสแตนซ์ไอดีไฟเดอร์ที่ประกอบไปด้วยค่าของ อ็อบเจกต์ไอดีไฟเดอร์ (Object Identifier) แล้วปิดท้ายด้วย 0 ก็จะอยู่ในรูปแบบของ

$$\text{Instance Identifier} = \text{Object Identifier}.0$$

เช่นการอ้างอิงถึงค่าในอ็อบเจกต์ sysDescr ด้วยค่าอินสแตนซ์ไอดีไฟเดอร์คือ 1.3.6.1.2.1.1.1.0 ซึ่งก็คือจะประกอบด้วยค่าอ็อบเจกต์ไอดีไฟเดอร์ของอ็อบเจกต์ sysDescr คือ 1.3.6.1.2.1.1.1 แล้วต่อท้ายด้วย 0 นอกจากนั้นยังอาจเขียนในรูปแบบของชื่อได้คือ iso.org.dod.internet.mgmt.mib-2.system.sysDescr.0 หรือเขียนสั้นๆเพียง sysDescr.0 ก็ได้

สำหรับการอ้างอิงถึงค่าของอ็อบเจกต์ในรูปแบบตาราง (Sequence of Object Value) นั้น เอสเอ็นเอ็มพีไม่อนุญาตให้อ้างอิงทั้งตารางได้ในคราวเดียว การอ้างอิงจะต้องทำที่อ็อบเจกต์ที่เป็นโหนดปลายเท่านั้น (Leaf object) หรือต้องเจาะจงถึงค่าในตารางเลขนั้นเอง ดังนั้นจึงต้องอาศัยเทคนิคเฉพาะในการอ้างอิงดังกล่าวซึ่งจะมีอยู่ 2 เทคนิคได้แก่ Serial-access technique และ Random-access technique สำหรับเทคนิคของ Random-access technique จะอธิบายไว้ในหัวข้อนี้ ส่วนเทคนิคของ Serial-access technique จะอธิบายไว้ในหัวข้อของคำสั่ง GetNextRequest ต่อไป

Random-access technique นั้นจะอาศัยแนวคิดที่ว่า สิ่งแยกความแตกต่างของแต่ละคอลัมน์ในตารางก็คืออ็อบเจกต์ไอดีไฟเดอร์ และสิ่งแยกความแตกต่างของแต่ละแถวในตารางก็คือค่าในอินเดกซ์อ็อบเจกต์ (Value of INDEX Object)

จากตัวอย่างตาราง tcpConnTable ดังรูปที่ 6 ซึ่งมีแบบข้อมูล sequence of และภายใต้ตาราง tcpConnTable มี tcpConnEntry ซึ่งมีแบบข้อมูลเป็น sequence โดยประกอบด้วย tcpConnState, tcpConnLocalAddress, tcpConnLocalPort, tcpConnRemAddress และ tcpConnRemPort และจะมีอินเดกซ์อ็อบเจกต์ (INDEX Object) คือ tcpConnLocalAddress, tcpConnLocalPort, tcpConnRemAddress และ tcpConnRemPort

การอ้างอิงถึงค่าของอ็อบเจกต์ในรูปแบบตารางตาม Random-access technique นี้ จะใช้อินสแตนซ์ไอดีไฟเดอร์ที่ประกอบไปด้วยการรวมกันของ อ็อบเจกต์ไอดีไฟเดอร์ (Object Identifier) และ ค่าในอินเดกซ์

อ็อบเจกต์ ก็จะอยู่ในรูปแบบของ

$$\text{Instance Identifier} = \text{Object Identifier} . (\text{ค่าในอินเดกซ์อ็อบเจกต์1}).(\text{ค่าในอินเดกซ์อ็อบเจกต์2}).(\text{ค่าในอินเดกซ์อ็อบเจกต์3})....(\text{ค่าในอินเดกซ์อ็อบเจกต์ N})$$

ดังนั้นจากตัวอย่างตาราง tcpConnTable จะใช้อินสแตนท์ไอเด็นติไฟเออร์ตามรูปแบบดังนี้คือ

x.i.(tcpConnLocalAddress).(tcpConnLocalPort).(tcpConnRemAddress).(tcpConnRemPort)

เมื่อ x = 1.3.6.1.2.1.6.13.1 ซึ่งคืออ็อบเจ็กต์ไอเด็นติไฟเออร์ของ tcpConnEntry
 i = เป็นหมายเลขที่ใช้ระบุเพื่อแยกความแตกต่างของแต่ละคอลัมน์ เช่น ถ้าต้องการ ระบุถึงคอลัมน์ tcpConnLocalAddress ก็ให้ค่าของอีกเท็ต (Octet) i เป็น 1 แต่ถ้าต้องการระบุถึงคอลัมน์ tcpConnLocalPort ก็ให้ค่าของอีกเท็ต (Octet) i เป็น 2 เป็นต้น ตามตัวเลขตัวสุดท้ายในอ็อบเจ็กต์ไอเด็นติไฟเออร์ของอ็อบเจ็กต์แต่ละตัว
 (name) = ค่าที่อยู่ในอินเดกซ์อ็อบเจ็กต์ เช่น (tcpConnLocalAddress) ก็จะหมายถึงค่าที่อยู่ในอ็อบเจ็กต์ tcpConnLocalAddress นั้นเอง

สำหรับอินสแตนท์ไอเด็นติไฟเออร์ของทั้งตาราง tcpConnTable ที่อาศัยค่าในอินเดกซ์อ็อบเจ็กต์จากตัวอย่างในรูป 7.1 จะแสดงได้ดังรูปที่ 4-8

tcpConnState (1.3.6.1.2.1.6.13.1.1)	tcpConnLocalAddress (1.3.6.1.2.1.6.13.1.2)	tcpConnLocalPort (1.3.6.1.2.1.6.13.1.3)	tcpConnRemAddress (1.3.6.1.2.1.6.13.1.4)	tcpConnRemPort (1.3.6.1.2.1.6.13.1.5)
x.1.10.0.0.99.12.9 .1.2.3.15	x.2.10.0.0.99.12. 9.1.2.3.15	x.3.10.0.0.99.12.9 .1.2.3.15	x.4.10.0.0.99.12. 9.1.2.3.15	x.5.10.0.0.99.12. 9.1.2.3.15
x.1.0.0.0.0.99. 0.0.0.0.0	x.2.0.0.0.0.99. 0.0.0.0.0	x.3.0.0.0.0.99. 0.0.0.0.0	x.4.0.0.0.0.99. 0.0.0.0.0	x.5.0.0.0.0.99. 0.0.0.0.0
x.1.10.0.0.99.14.8 9.1.1.24.84	x.2.10.0.0.99.14. 89.1.1.42.84	x.3.10.0.0.99.14.8 9.1.1.42.84	x.4.10.0.0.99.14. 89.1.1.42.84	x.5.10.0.0.99.14. 89.1.1.42.84

รูปที่ 4-8 อินสแตนท์ไอเด็นติไฟเออร์ของทั้งตาราง tcpConnTable เมื่อ x = 1.3.6.1.2.1.6.13.1 ซึ่งคืออ็อบเจ็กต์ไอเด็นติไฟเออร์ของ tcpConnEntry

เนื่องจากชนิดของ Object มีมากมายดังนั้นจึงมีการตั้งกฎเกณฑ์ในการนำเอาค่าในอินเดกซ์อ็อบเจ็กต์ที่จะเอามาประกอบเป็นอินสแตนท์ไอเด็นติไฟเออร์ดังนี้

- ค่าของ Integer (Integer-valued): ให้นำค่าของ Integer ดังกล่าวมาประกอบรวมเป็น 1 อ็อกเท็ต (Octet) ของอินสแตนท์ไอเด็นติไฟเออร์ได้ทันที

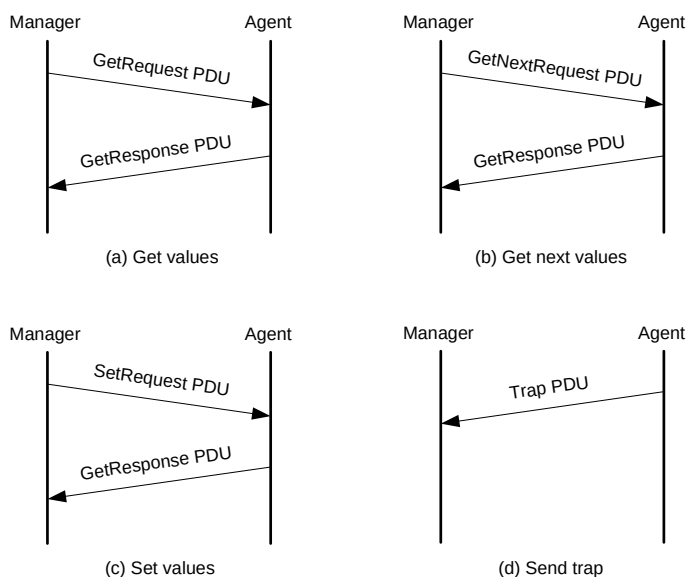
- ค่าของ String ที่มีความยาวคงที่ (String-valued, fixed-length): ให้นำค่าของตัวอักษรในแต่ละอักขระที่ตามมาแปลงเป็นตัวเลขในแต่ละอักขระที่ติดกันในอินสแตนท์ไอดีเอ็นดีไฟเบอร์ เช่น ถ้า String นั้นมีความยาว N ตัวอักษรก็หมายความว่า ค่าในอินเดกซ์อ็อบเจกต์แปลงเป็นตัวเลขได้ N อักขระ
- ค่าของ String ที่มีความยาวไม่คงที่ (String-valued, variable-length): ให้นำค่าของตัวอักษรในแต่ละอักขระที่ตามมาแปลงเป็นตัวเลขในแต่ละอักขระที่ติดกันในอินสแตนท์ไอดีเอ็นดีไฟเบอร์และต่อท้ายด้วยค่าของจำนวนตัวอักษรทั้งหมดอีกหนึ่งอักขระที่ติด เช่น ถ้า String นั้นมีความยาว N ตัวอักษรก็หมายความว่า ค่าในอินเดกซ์อ็อบเจกต์แปลงเป็นตัวเลขได้ N+1 อักขระ โดยอักขระที่ N+1 จะคือค่าของจำนวนตัวอักษรทั้งหมด
- ค่าของอ็อบเจกต์ไอดีเอ็นดีไฟเบอร์ (Object-identifier-valued): ค่าในแต่ละอักขระของอ็อบเจกต์ไอดีเอ็นดีไฟเบอร์จะกลายเป็นค่าตัวเลขในแต่ละอักขระที่ติดกันในอินสแตนท์ไอดีเอ็นดีไฟเบอร์โดยอัตโนมัติและต่อท้ายด้วยค่าของจำนวนอักขระทั้งหมดอีกหนึ่งอักขระที่ติด เช่น ถ้าค่าของอ็อบเจกต์ไอดีเอ็นดีไฟเบอร์มีความยาว N อักขระก็หมายความว่า ค่าในอินเดกซ์อ็อบเจกต์แปลงเป็นตัวเลขได้ N+1 อักขระ โดยอักขระที่ N+1 จะคือค่าจำนวนอักขระของอ็อบเจกต์ไอดีเอ็นดีไฟเบอร์ทั้งหมดที่มี
- ค่าของ IP address (IpAddress-valued): ค่าของ IP Address จะแปลงเป็น 4 อักขระของอินสแตนท์ไอดีเอ็นดีไฟเบอร์ได้ทันที

4.7 ลักษณะของโปรโตคอล (Protocol Specification)

การติดต่อระหว่างสถานีจัดการกับเอเจนต์มีรูปแบบในการติดต่อที่เรียกว่า protocol data units หรือ พีดียู (PDU) หลายรูปแบบด้วยกันตามวัตถุประสงค์ในการติดต่อ แบบของการติดต่อในเอสเอ็นเอ็มพีรุ่น 1 มี 5 แบบคือ

- 4.7.1 GetRequest PDU ใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่าย
- 4.7.2 GetNextRequest PDU ใช้สอบถามข้อมูลที่เรียงเป็นลำดับ เช่นข้อมูลที่เก็บอยู่ในรูปตารางหรือในกรณีที่ไม่ทราบชื่อตัวแปรที่แน่ชัด
- 4.7.3 GetResponse PDU เอเจนต์ส่งคำตอบกลับมายังผู้สอบถาม
- 4.7.4 SetRequest PDU ใช้เปลี่ยนแปลงค่าของอ็อบเจกต์ที่เอเจนต์รับผิดชอบอยู่
- 4.7.5 Trap PDU ใช้แจ้งเหตุการณ์ที่เกิดขึ้นในระบบเครือข่าย เช่นการเริ่มต้นทำงานใหม่ของอุปกรณ์ หรือเส้นทางขัดข้อง

เอสเอ็นเอ็มพีอาศัยโปรโตคอลยูดีพี โดยเอเจนต์จะใช้พอร์ตหมายเลข 161 สำหรับการติดต่อในแบบที่ 1 ถึง 3 จากสถานีจัดการเครือข่ายและสถานีจัดการเครือข่ายจะใช้พอร์ตหมายเลข 162 สำหรับการติดต่อในแบบที่ 5 จากเอเจนต์ โดยจะมีลำดับการทำงานในการติดต่อทั้ง 5 รูปแบบดังรูปที่ 4-9

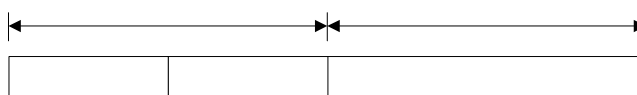


รูปที่ 4-9 ลำดับการทำงานของ SNMP PDU

4.7.1 การเ็นแคปซูล (Encapsulation)

การเ็นแคปซูลคำสั่งและข้อมูลในเอสเอ็นเอ็มพีมีวิธีตามรูปที่ 4-10 พอร์แมตของเอสเอ็นเอ็มพีประกอบด้วย 2 ส่วนคือ เฮดเดอร์และพิดิยู เฮดเดอร์ประกอบด้วยฟิลด์ย่อยสองฟิลด์คือ

- Version รุ่นของโพรโตคอลที่ใช้ ถ้าเป็นโพรโตคอลรุ่น 1 จะมีค่า 0 หากเป็นรุ่น 2 จะมีค่า 1
- Community รหัสผ่านในรูปสายอักขระเพื่อให้เอเจนต์ใช้ในการพิสูจน์ตัวตนว่าข้อความที่ส่งมามีสิทธิ์ในการสอบถามหรือเปลี่ยนแปลงข้อมูลหรือไม่ ซึ่งรายละเอียดได้อธิบายไว้ในหัวข้อ Communities and Community Names ที่ผ่านมา



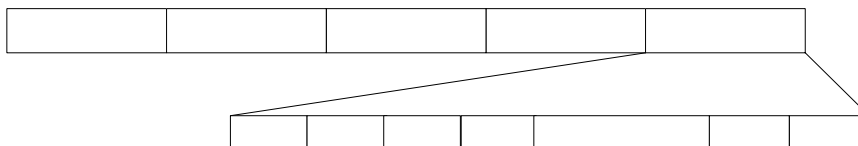
รูปที่ 4-10 การเ็นแคปซูลเอสเอ็นเอ็มพี

ในส่วนของพิดิยูประกอบด้วยฟิลด์ย่อยตามชนิดของข้อความ หากเป็นข้อความ *GetRequest PDU*, *GetNextRequest PDU*, *GetResponse PDU* และ *SetRequest PDU* จะมีโครงสร้างเดียวกัน รูปที่ 4-11 แสดงโครงสร้างของพิดิยูโดยแต่ละฟิลด์มีความหมายดังนี้

- PDU type รูปแบบการติดต่อ (1 ถึง 5)
- Request ID กำหนดบอกหมายเลขข้อความเพื่อใช้จับคู่เมื่อรับคำตอบกลับมา
- Error Status สถานะความผิดพลาดที่เกิดขึ้น โดยรหัสความผิดพลาดและสถานะผิดพลาดที่ใช้ในเอสเอ็นเอ็มพีจะแสดงในตารางที่ 3 สำหรับข้อความ

GetRequest PDU, GetNextRequest PDU และ SetRequest PDU จะมีค่าในฟิลด์นี้เป็น 0 เสมอ

- Error Index กรณีที่ค่าผิดพลาดที่เกิดขึ้นเกิดจากตัวแปรตัวลำดับที่เท่าใดหนึ่งของตัวแปรทั้งหมดที่สอบถามไปสำหรับข้อความ GetRequest PDU, GetNextRequest PDU และ SetRequest PDU จะมีค่าในฟิลด์นี้เป็น 0 เสมอ
- VarBindList ค่าผูกพันตัวแปร(variable binding) แสดงอยู่ในรูปของการอ้างอิงตัวแปรหรืออ็อบเจ็กต์ (name) และค่าของตัวแปร (value) ต่อเนื่องกันไปเป็นรายการสำหรับข้อความ GetRequest PDU, GetNextRequest PDU และ SetRequest PDU ค่าของตัวแปรจะมีค่าเป็น NULL เสมอ



รูปที่ 4-11 โครงสร้างฟิลด์ของ GetRequest PDU, GetNextRequest PDU, GetResponse PDU และ SetRequest PDU

รหัสผิดพลาด	ชื่อ	คำอธิบาย
0	noError	ไม่มีข้อผิดพลาด
1	tooBig	เอเจนต์ไม่สามารถส่งคำตอบได้ในเฟรมเดียว
2	noSuchName	ไม่มีตัวอ็อบเจ็กต์ที่ต้องการสอบถามอยู่ในฐานข้อมูล
3	badValue	ค่าที่กำหนดให้อ็อบเจ็กต์ไม่ถูกต้อง
4	readOnly	เปลี่ยนค่าอ็อบเจ็กต์ไม่ได้เพราะอ่านค่าได้เพียงอย่างเดียว
20	genErr	มีข้อผิดพลาดอื่นๆเกิดขึ้น

ตารางที่ 4-3 รหัสและสถานะความผิดพลาดในเอสเอ็นเอ็มพี

สำหรับข้อความ Trap PDU มีลักษณะแตกต่างกันออกไปดังรูปที่ 4-12 โดยแต่ละฟิลด์มีความหมายดังนี้

- Enterprise ชนิดของตัวแปรที่สร้าง Trap นี้ขึ้นมา โดยค่าในฟิลด์นี้จะอ้างอิงกับค่าในอ็อบเจ็กต์ sysObjectID
- Agent Addr ค่า IP Address ของอ็อบเจ็กต์ที่สร้าง Trap นี้ขึ้นมา
- Generic Trap ชนิดของ Trap ที่เกิดขึ้นโดยชนิดของ Trap และรหัสของ Trap ที่ใช้ในเอสเอ็นเอ็มพีจะแสดงในตารางที่ 4-4

- Specific Trap ชนิดของเหตุการณ์ผิดปกติเกิดขึ้นกับ enterprise-specific
- Time-stamp เวลาที่ใช้ไปทั้งหมดตั้งแต่การเริ่มการติดต่อในเครือข่ายรวมถึงเวลาที่ใช้ในการสร้าง Trap โดยค่าดังกล่าวจะเก็บอยู่ในอ็อบเจ็กต์ sysUpTime

รหัส Trap	ชื่อ	คำอธิบาย
0	coldStart	มีการเริ่มต้นการทำงานใหม่ ด้วยสาเหตุของการเปลี่ยนแปลงค่าในเอเจนต์ หรือมีการเปลี่ยนแปลงโปรโตคอลเอ็นจิน เช่น การ restart อุปกรณ์หลังจากเกิดการล้มเหลวของระบบ (crash)
1	warmStart	มีการเริ่มต้นการทำงานใหม่ แต่ไม่ได้เกิดจากสาเหตุของการเปลี่ยนแปลงค่าในเอเจนต์ หรือโปรโตคอลเอ็นจิน เช่น การ restart routine
2	linkDown	การเชื่อมต่อของเอเจนต์มีปัญหา จะมีการค่าของอ็อบเจ็กต์ ifIndex เพื่อบอกจุดเชื่อมต่อ (interface) ที่มีปัญหา
3	linkUp	การเชื่อมต่อของเอเจนต์กลับมาใช้งานได้ จะมีการค่าของอ็อบเจ็กต์ ifIndex เพื่อบอกจุดเชื่อมต่อ (interface) ที่มีการกลับมาใช้งานได้
4	authenticationFailure	การพิสูจน์ตัวตนของ message ที่เข้ามาไม่ผ่านหรือมีการผิดพลาดเกิดขึ้น
5	egpNeighborLoss	โปรโตคอลเกตเวย์ภายนอก (External gateway protocol) มีปัญหา
6	enterpriseSpecific	มีเหตุการณ์ผิดปกติเกิดขึ้นกับ enterprise-specific โดยจะมีการประกาศชนิดของเหตุการณ์ผิดปกติเกิดขึ้นในฟิลด์ specific-trap

ตารางที่ 4-4 รหัสและชนิดของ Trap ในเอสเอ็นเอ็มพี



รูปที่ 4-12 โครงสร้างพิดิยูของคำสั่ง Trap PDU

โปรดสังเกตว่าในที่นี้ไม่ได้กล่าวขนาดความยาวของแต่ละฟิลด์ เพราะทุกฟิลด์ในเอสเอ็นเอ็มพีต้องเข้ารหัสและจะได้ขนาดของแต่ละฟิลด์ที่มีความยาวแตกต่างกันไปตามชนิดข้อมูล

4.7.2 กลไกในการส่ง SNMP Message (Transmission of an SNMP Message)

กลไกในการส่ง SNMP Message ของ PDU ทั้ง 5 แบบในส่วนของโปรโตคอลเอ็นจิน โดยการทำงานจะมีขั้นตอนดังต่อไปนี้

- 4.7.2.1 สร้าง PDU ตามโครงสร้างที่กำหนดไว้ใน ASN.1 (Abstract Syntax Notation One)
- 4.7.2.2 PDU ที่สร้างในขั้นตอนที่หนึ่งรวมค่า transport addresses ของต้นทางและปลายทาง และ Community name จะถูกส่งไปให้ส่วนของการบริการพิสูจน์ตัวตน (Authentication service) ซึ่งจะทำการปรับเปลี่ยนข้อมูลที่จำเป็นในการแลกเปลี่ยน เช่น การเข้ารหัสลับ (encryption) หรือการสร้างโค้ดที่ใช้ในการพิสูจน์ตัวตน หลังจากการปรับเปลี่ยนเสร็จก็จะคืนค่ากลับมายังโปรโตคอลเอ็นจิน
- 4.7.2.3 โปรโตคอลเอ็นจินจะสร้าง message ของเอสเอ็นเอ็มพีโดยการรวมฟิลด์รุ่นของโปรโตคอล (Version), Community name, และผลลัพธ์ที่ได้ในขั้นตอนที่ 2
- 4.7.2.4 ทำการเข้ารหัส message เอสเอ็นเอ็มพีที่ได้จากขั้นตอนที่ 3 โดยวิธีการ Basic Encoding Rule (BER) แล้วส่ง message ที่เข้ารหัสแล้วไปยังโปรโตคอลในชั้น Transport ต่อไป

4.7.3 กลไกในการรับ SNMP Message (Receipt of an SNMP Message)

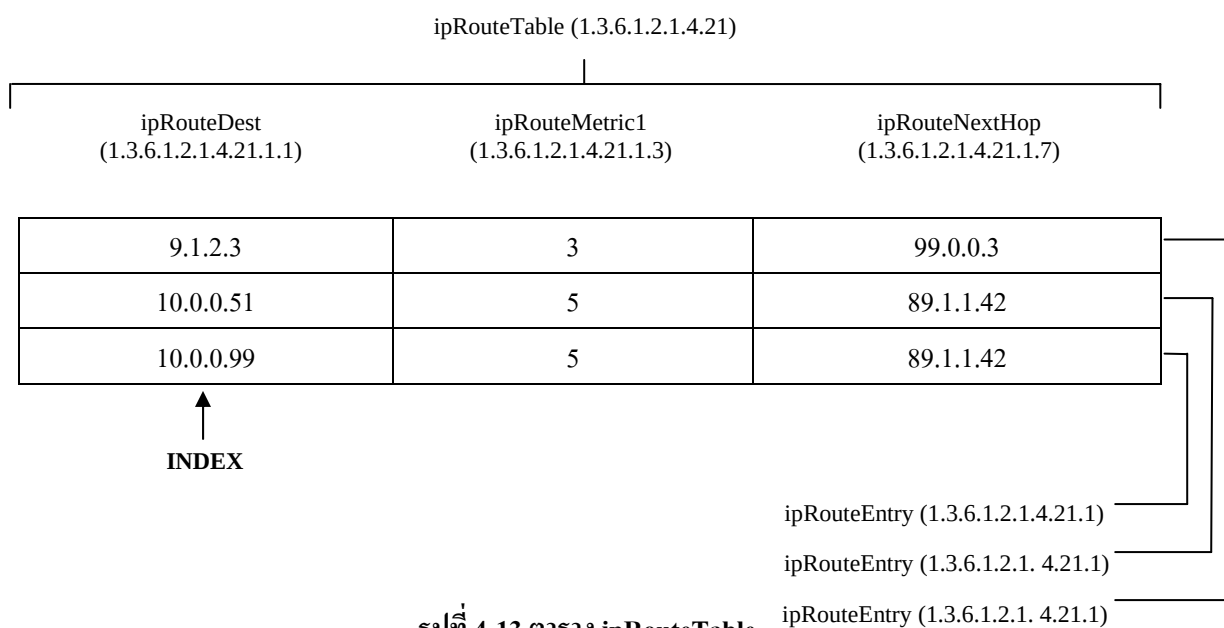
กลไกในการรับ SNMP Message ของ PDU ในส่วนของโปรโตคอลเอ็นจิน โดยการทำงานจะมีขั้นตอนดังต่อไปนี้

- 4.7.3.1 ตรวจสอบความถูกต้องทางไวยากรณ์ขั้นพื้นฐานของ message โดยจะกำจัด message ที่ถ้าพบความผิดพลาด
- 4.7.3.2 รุ่นของโปรโตคอล (Version) ที่ใช้ โดยจะกำจัด message ที่ถ้าพบว่าเป็นคนละรุ่นกัน
- 4.7.3.3 โปรโตคอลเอ็นจินจะนำค่าของ use name, ส่วนของ PDU และ transport addresses ของต้นทางและปลายทาง ส่งไปให้ส่วนของการบริการพิสูจน์ตัวตน (Authentication service)
 - ถ้าการพิสูจน์ตัวตนล้มเหลว ส่วนของการบริการพิสูจน์ตัวตนจะส่งสัญญาณบอกไปยังโปรโตคอลเอ็นจินเพื่อให้ทำการสร้าง Trap และจะกำจัด message นั้นทิ้ง
 - ถ้าการพิสูจน์ตัวตนสำเร็จ ส่วนของการบริการพิสูจน์ตัวตนจะคืนค่า PDU ที่อยู่ในโครงสร้างของ ASN.1 มาให้กับโปรโตคอลเอ็นจิน
- 4.7.3.4 ตรวจสอบความถูกต้องทางไวยากรณ์ขั้นพื้นฐานของ PDU โดยจะกำจัด PDU ที่ถ้าพบความผิดพลาด หลังจากนั้นจะนำชื่อของ Community name เพื่อจัดสรรรูปแบบนโยบายการเข้าถึง (Access policy) ตาม Community ของ PDU นั้น และทำการประมวลผลตามคำสั่งใน PDU ต่อไป

4.7.4 GetRequest PDU

GetRequest PDU จะถูกส่งจากสถานีจัดการเครือข่ายเพื่อใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่าย โดยจะมีลำดับขั้นตอนการทำงานดังรูปที่ 8 (a)

เอสเอ็นเอ็มพีจะอนุญาตให้การอ้างอิงกระทำได้ที่โหนดปลายทางนั้น และไม่อนุญาตอ้างอิงได้ทั้งโครงสร้างของตารางในคราวเดียวดังที่กล่าวในหัวข้อที่ผ่านมา อย่างไรก็ตามสถานีจัดการเครือข่ายสามารถที่จะอ้างอิงถึงค่าในทั้งแถวของตารางพร้อมกันได้ โดยอาศัยคุณสมบัติของฟิลด์ VarBindList ซึ่งสามารถผูกพันค่าของตัวแปรเป็นรายการได้



รูปที่ 4-13 ตาราง ipRouteTable

ตัวอย่างของตาราง ipRouteTable ดังรูปที่ 12 สถานีจัดการเครือข่ายสามารถสอบถามค่าของอ็อบเจ็กต์ได้ทั้งแถวของตารางโดยใช้รูปแบบของการติดต่อคือ

GetRequest (ipRouteDest.9.1.2.3, ipRouteMetric1.9.1.2.3, ipRouteNextHop.9.1.2.3)

สำหรับในบางกรณีของ Object มีขนาดใหญ่มากและมีการสอบถามค่ามาทีละหลายอ็อบเจ็กต์เกินกว่าที่ GetResponse PDU เพียง message เดียวจะตอบกลับมาได้ อาจจะเป็นผลให้ไม่มีการคืนค่าของข้อมูลกลับมาเลยโดยอาจมีการส่งค่าสถานะความผิดมาเป็น tooBig ได้

4.7.5 GetNextRequest PDU

GetNextRequest PDU จะถูกส่งจากสถานีจัดการเครือข่ายเพื่อใช้สอบถามข้อมูลที่เรียงเป็นลำดับ เช่นข้อมูลที่เก็บอยู่ในรูปตาราง หรือในกรณีที่ไม่ทราบชื่อตัวแปรที่แน่ชัด โดยจะมีลำดับขั้นตอนการ

ทำงานดังรูปที่ 8 (b) ประโยชน์ของการใช้การติดต่อแบบ GetNextRequest PDU คือจะสามารถใช้ในการ ค้นหาโครงสร้างของฐานข้อมูลสารสนเทศการจัดการได้อย่างมีประสิทธิภาพ สำหรับรูปแบบของการ สอบถามข้อมูลของ GetNextRequest PDU จะมีรูปแบบของการสอบถามข้อมูล 2 รูปแบบคือ

4.7.5.1 การสอบถามค่าของอ็อบเจ็กแบบปกติ (Simple Object Value)

สำหรับการสอบถามค่าของอ็อบเจ็กแบบปกติ จากตัวอย่างถ้าสถานีจัดการเครือข่ายต้องการ สอบถามค่าของอ็อบเจ็กแบบที่ไม่ใช่ตารางทั้งหมดในกลุ่มของ udp สถานีจัดการเครือข่ายจะส่งการติดต่อ แบบ GetRequest ซึ่งระบุค่าการอ้างอิงของอินสแตนซ์ไอเด็นติไฟเออร์ คือ

```
GetRequest (udpInDatagrams.0, udpNoPorts.0, udpInErrors.0,
            udpOutDatagrams.0)
```

ถ้าในกรณีนี้ Community ที่อยู่ในเอเจนต์สนับสนุนอ็อบเจ็กเหล่านี้ทุกตัว เอเจนต์จะส่ง GetResponse PDU กลับมาในรูปแบบ

```
GetResponse ((udpInDatagrams.0 = 100), (udpNoPorts.0 = 1),
            (udpInErrors.0 = 2), (udpOutDatagrams.0 = 200))
```

ค่าของ 100, 1, 2 และ 200 คือค่าของอ็อบเจ็กทั้ง 4 ที่สอบถามไป แต่อย่างไรก็ตามถ้าเกิดใน กรณีที่อ็อบเจ็กตัวใดตัวหนึ่งหรือหลายตัวไม่มีตัวตน ทำให้ GetResponse PDU ที่ส่งกลับมามีค่าสถานะ ความผิดพลาดเป็น noSuchName และไม่มีการคืนค่าที่สอบถามใดๆมา

แต่สำหรับกลไกการทำงานของการทำงานของการติดต่อแบบ GetNextRequest PDU คือเมื่อเอเจนต์ได้รับ GetNextRequest PDU ซึ่งระบุค่าของอ็อบเจ็กไอเด็นติไฟเออร์ไม่ใช่อินสแตนซ์ไอเด็นติไฟเออร์ ในกรณีนี้ ค่าที่ได้รับกลับมามีค่าที่เก็บอยู่ในอ็อบเจ็กที่อ้างอิงตามอ็อบเจ็กไอเด็นติไฟเออร์ในกรณีถ้าอ็อบเจ็กนั้น มีตัวตน แต่ถ้าในกรณีที่อ็อบเจ็กที่สอบถามไปนั้นไม่มีตัวตนค่าที่ส่งคืนกลับมาแทนคือค่าที่เก็บอยู่ในอ็อบเจ็กที่อ้างอิงตาม

อ็อบเจ็กไอเด็นติไฟเออร์ตัวถัดไป ซึ่งกลไกการทำงานดังกล่าวจะมีประสิทธิภาพกว่าการติดต่อในแบบ GetRequest

ถ้าตัวอย่างที่ผ่านมาถ้าสถานีจัดการเครือข่ายจะอาศัย GetNextRequest PDU โดยส่งการติดต่อ ในรูปแบบของ

```
GetNextRequest (udpInDatagrams, udpNoPorts, udpInErrors, udpOutDatagrams)
```

ในกรณีนี้จะได้รับ GetResponse PDU ตอบกลับมามีค่าคือ

GetResponse ((udpInDatagrams.0 = 100), (udpNoPorts.0 = 1),
(udpInErrors.0 = 2), (udpOutDatagrams.0 = 200))

และถ้าเกิดในกรณีที่อ็อบเจ็กต์ udpNoPorts ไม่มีตัวตนทำให้ค่าที่ได้รับคืนกลับมาแทนส่วนค่าของ
อ็อบเจ็กต์ udpNoPorts ก็คือค่าที่เก็บอยู่ในอ็อบเจ็กต์ที่อ้างอิงตามอ็อบเจ็กต์ไออน์ดิไฟเออร์ตัวถัดไปซึ่งก็คือค่า
ใน
อ็อบเจ็กต์ udpInErrors โดยจะได้รับ GetResponse PDU ตอบกลับมาคือ

GetResponse ((udpInDatagrams.0 = 100), (udpInErrors.0 = 2),
(udpInErrors.0 = 2), (udpOutDatagrams.0 = 200))

4.7.5.2 การสอบถามค่าของอ็อบเจ็กต์ในรูปแบบตาราง (Sequence of Object Value)

สำหรับการสอบถามค่าของอ็อบเจ็กต์ในรูปแบบตาราง โดยใช้การติดต่อแบบ GetNextRequest PDU นั้นจะอาศัยเทคนิคของ Serial-access technique เนื่องจากการติดต่อแบบ GetNextRequest จะสามารถจะค่าถัดไปมาได้โดยไม่ต้องเจาะจงค่าถัดไปโดยตรงซึ่งจะสะดวกมาต่ออ็อบเจ็กต์ที่เป็นชนิด sequence และ sequence of โดยลำดับการนำค่าในแต่ละอ็อบเจ็กต์ของ GetNextRequest จะมีลำดับเริ่มจากคอลลัมน์แรกไปจนสิ้นสุดทุกแถวในคอลลัมน์นั้นก่อนที่จะขึ้นคอลลัมน์ถัดไป

ตัวอย่างเช่นถ้าสถานีจัดการเครือข่ายต้องการสอบถามค่าของอ็อบเจ็กต์ที่อยู่ในตาราง ipRouteTable ดังในรูปที่ 12 ที่ในหัวข้อผ่านมาโดยจะเริ่มที่ GetNextRequest PDU ซึ่งระบุค่าของอ็อบเจ็กต์ไออน์ดิไฟเออร์ของอ็อบเจ็กต์ในแต่ละคอลลัมน์ดังนี้

GetNextRequest (ipRouteDest, ipRouteMetric1, ipRouteNextHop)

ซึ่งจะได้รับการตอบกลับจากเอเจนต์คือค่าของอ็อบเจ็กต์ในตารางแถวแรกดังนี้

GetResponse ((ipRouteDest.9.1.2.3 = 9.1.2.3), (ipRouteMetric1.9.1.2.3 = 3),
(ipRouteNextHop.9.1.2.3 = 99.0.0.3))

สถานีจัดการเครือข่ายจะนำค่าที่ได้รับการตอบรับดังกล่าวมาใช้ในการอ้างอิงโดยใช้อินสแตนซ์ไออน์ดิไฟเออร์เพื่อสอบถามข้อมูลในแถวที่สองของตารางดังนี้

GetNextRequest (ipRouteDest.9.1.2.3, ipRouteMetric1.9.1.2.3,
ipRouteNextHop.9.1.2.3)

ซึ่งจะได้รับการตอบกลับจากเอเจนต์คือค่าของอ็อบเจ็กต์ในตารางแถวสองดังนี้

```
GetResponse ((ipRouteDest.10.0.0.51 = 10.0.0.51),
(ipRouteMetric1.10.0.0.51 = 5), (ipRouteNextHop.10.0.0.51 = 89.1.1.42))
```

และสถานีจัดการเครือข่ายจะนำค่าที่ได้รับการตอบรับดังกล่าวมาใช้ในการอ้างอิงโดยใช้อินสแตนท์ไอดีเอ็นดีไฟเบอร์เพื่อสอบถามข้อมูลในแถวที่สามของตารางและได้รับการตอบกลับจากเอเจนต์ดังนี้

```
GetNextRequest (ipRouteDest.10.0.0.51, ipRouteMetric10.0.0.51,
ipRouteNextHop.10.0.0.51)
```

```
GetResponse ((ipRouteDest.10.0.0.99 = 10.0.0.99), (ipRouteMetric1.10.0.0.99 = 5),
(ipRouteNextHop.10.0.0.99 = 89.1.1.42))
```

เนื่องจากสถานีจัดการเครือข่ายไม่ทราบว่าข้อมูลในตารางหมดแล้ว จึงนำค่าที่ได้รับการตอบรับในแถวที่สามดังกล่าวมาใช้ในการอ้างอิงโดยใช้อินสแตนท์ไอดีเอ็นดีไฟเบอร์เพื่อสอบถามข้อมูลในแถวที่สี่และได้รับการตอบกลับมาคือ

```
GetNextRequest (ipRouteDest.10.0.0.99, ipRouteMetric10.0.0.99,
ipRouteNextHop.10.0.0.99)
```

```
GetResponse ((ipRouteMetric1.9.1.2.3 = 3), (ipRouteNextHop.9.1.2.3 = 99.0.0.3),
(ipNetToMediaIfIndex.1.3 = 1))
```

จะเห็นได้ว่าค่าของอ็อบเจ็กต์ที่ได้รับการตอบกลับจากเอเจนต์เป็นการขึ้นคอลัมน์ถัดไป ซึ่งก็จะหมายความว่าได้สิ้นสุดแถวของตารางแล้วนั่นเอง

4.7.6 SetRequest PDU

ใช้เปลี่ยนแปลงค่าของอ็อบเจ็กต์ที่เอเจนต์รับผิดชอบอยู่ ซึ่งฟิลด์ของ VarBindList จะประกอบด้วยค่าอ้างอิงของตัวแปรหรืออ็อบเจ็กต์ และค่าของอ็อบเจ็กต์เหล่านั้นซึ่งค่าเหล่านี้จะเป็นค่าที่นำไปใช้เปลี่ยนแปลงค่าตัว

อ็อบเจ็กต์ในฐานข้อมูลสารสนเทศการจัดการที่เอเจนต์รับผิดชอบอยู่ สำหรับลำดับขั้นตอนการทำงานจะแสดงดังรูปที่ 8 (c)

สำหรับการใช้งานของ SetRequest PDU กับค่าในอ็อบเจ็กต์ที่อยู่ในรูปแบบตาราง นอกเหนือจากการเปลี่ยนแปลง (Update) ค่าในตาราง ยังจะสามารถที่จะเพิ่มแถว (Insert row) หรือลบ (Delete) ค่าในตารางโดยจะอาศัยตัวอย่างจากตาราง ipRouteTable ในรูปที่ 12 ซึ่งมีรายละเอียดดังนี้

4.7.6.1 การเปลี่ยนแปลงค่าในตาราง (Update a Table) SetRequest PDU สามารถที่การเปลี่ยนแปลงค่าในตารางได้โดยการอ้างอิงถึงค่าของอ็อบเจ็กต์จะอาศัยการอ้างอิงของอินสแตนท์ไอดีเอ็นดีไฟเออร์ เช่นเดียวกับ

อ็อบเจ็กต์ปกติทั่วไปยกตัวอย่างเช่น ถ้าสถานีจัดการเครือข่ายส่งการติดต่อในรูปแบบ

SetRequest (ipRouteMetric1.9.1.2.3 = 9)

และเอเจนต์จะส่งการติดต่อกลับคืนมาในรูปแบบ

GetResponse (ipRouteMetric1.9.1.2.3 = 9)

การติดต่อดังกล่าวคือสถานีจัดการเครือข่ายได้ส่ง SetRequest PDU เพื่อทำการเปลี่ยนแปลงค่าของอ็อบเจ็กต์ ipRouteMetric1 ที่อยู่ในแถวแรกเนื่องจากในส่วนของการอ้างอิงได้ระบุค่าในอินเดกซ์อ็อบเจ็กต์ของแถวแรกไว้คือ 9.1.2.3 และเมื่อเอเจนต์ได้รับก็จะทำการเปลี่ยนแปลงค่าในอ็อบเจ็กต์ ipRouteMetric1 จาก 3 เป็น 9 แล้วจะส่ง GetResponse กลับไปให้สถานีจัดการเครือข่ายเพื่อยืนยันการเปลี่ยนแปลง

4.7.6.2 การเพิ่มแถวในตาราง (Insert row to a Table) ถ้าสถานีจัดการเครือข่ายต้องการที่จะเพิ่มแถวในตาราง ipRouteTable โดยกำหนดค่าในแต่ละอ็อบเจ็กต์ ipRouteDest, ipRouteMetric1 และ ipRouteNextHop เป็น 11.3.3.12, 9 และ 91.0.0.5 ตามลำดับ สถานีจัดการเครือข่ายส่งการติดต่อในรูปแบบ

SetRequest ((ipRouteDest.11.3.3.12 = 11.3.3.12), (ipRouteMetric1.11.3.3.12 = 9),
(ipRouteNextHop.11.3.3.12 = 91.0.0.5))

ซึ่งการเปลี่ยนแปลงค่าดังกล่าวจะเห็นได้ว่าไม่มีค่าของอ็อบเจ็กต์ ipRouteDest ซึ่งเป็นอินเดกซ์อ็อบเจ็กต์ในแถวใดเลยที่มีค่าเป็น 11.3.3.12 ใน RFC 1212 จะมี 3 ทางเลือกให้เอเจนต์จัดการกับเหตุการณ์นี้คือ

- เอเจนต์ทำการปฏิเสธการทำงานของ SetRequest PDU และทำการส่ง GetResponse PDU ที่มีสถานะความผิดพลาดเป็น noSuchName
- เอเจนต์จะทำการยอมรับการทำงานของ SetRequest PDU แต่ถ้าเกิดพบว่าค่าที่ส่งมาให้เปลี่ยนแปลงนั้นผิดหลักไวยากรณ์หรือมีชนิดของอ็อบเจกต์ผิดไปจากที่ได้กำหนดไว้ ก็จะไม่มีการสร้างแถวใหม่ในตารางและทำการส่ง GetResponse PDU ที่มีสถานะความผิดพลาดเป็น badValue
- เอเจนต์จะทำการยอมรับการทำงานของ SetRequest PDU และเพิ่มแถวใหม่ลงในตาราง ถ้าสมมุติว่าเอเจนต์จัดการตามทางเลือกที่ 3 คือทำการสร้างแถวใหม่ให้ในตาราง และทำการส่ง GetResponse กลับคืนมาคือ

GetResponse ((ipRouteDest.11.3.3.12 = 11.3.3.12), (ipRouteMetric1.11.3.3.12 = 9),
(ipRouteNextHop.11.3.3.12 = 91.0.0.5))

แต่ถ้าในกรณีที่สถานีจัดการเครือข่ายต้องการที่จะเพิ่มแถวในตารางแต่ส่งค่ามากับ SetRequest PDU เพียงบางค่า คือ

SetRequest ((ipRouteDest.11.3.3.12 = 11.3.3.12))

จะมีสองทางเลือกที่เอเจนต์จะจัดการกับเหตุการณ์ดังกล่าวได้แก่

- เอเจนต์จะเพิ่มแถวใหม่ลงไปตารางแต่ค่าในอ็อบเจกต์ที่ SetRequest ไม่ได้ส่งมาจะถูกกำหนดเป็นค่าโดยปริยาย (Default values)
- เอเจนต์ทำการปฏิเสธการทำงานของ SetRequest PDU โดยในกรณีนี้เอเจนต์จะต้องการให้ SetRequest ส่งค่าของอ็อบเจกต์ทุกตัวในแถวในกรณีที่ต้องการเพิ่มแถวใหม่

4.7.6.3 การลบแถวในตาราง (Delete row in a Table) คำสั่ง set ยังสามารถใช้ลบแถวในตารางได้ เช่น สถานีจัดการเครือข่ายส่ง SetRequest PDU และได้รับการตอบกลับจากเอเจนต์เป็น

SetRequest (ipRouteNextHop.10.0.0.51 = invalid)

GetResponse (ipRouteDest.10.0.0.51 = invalid)

เอเจนต์เมื่อได้รับ SetRequest มาเพื่อเปลี่ยนแปลงค่าของ ipRouteNextHop ในแถวที่มีค่าของอินเดกซ์อ็อบเจกต์คือ ipRouteDest เป็น 10.0.0.51 ให้มีค่าเป็น invalid ซึ่งก็จะมีผลหมายความว่าให้ลบค่าของแถวที่มีค่าของอินเดกซ์อ็อบเจกต์คือ ipRouteDest เป็น 10.0.0.51 นั้นเอง และทำ

การส่ง GetResponse และที่ระบุค่าของอินเดกซ์ที่จับในแถวที่ลบเป็น invalid เพื่อทำการยืนยัน

4.8 การเข้ารหัสโดยใช้ BER (Basic Encoding Rules)

การส่งข้อมูลใน SNMP ไม่ได้ส่งในรูปแบบเป็น ออกเขตของของตัวเลขโดยตรง หากแต่ฝ่ายส่งต้องเข้ารหัสข้อมูลเพื่อนำส่งข้อมูลออก และ ถอดรหัสที่ฝ่ายรับ

4.8.1 โครงสร้างการเข้ารหัส

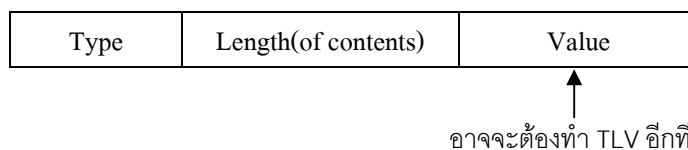
การเข้ารหัสตามแบบ BER จะมีข่าวสารกำกับอยู่ในตัวว่าเป็นข้อมูลชนิดใด และ มีความยาวเท่าใด ฟิลด์ที่ผ่านการเข้ารหัสแล้วประกอบด้วยค่า 3 ส่วน คือ

4.8.1.1 ชนิดของข้อมูล (type หรือ tag หรือ identifier)

4.8.1.2 ความยาวของข้อมูล (length)

4.8.1.3 ตัวข้อมูล (value หรือ contents)

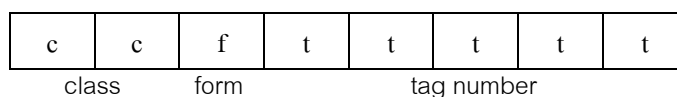
โครงสร้างรหัสเหล่านี้ เรียกว่า โครงสร้าง TLV (TLV : type-length-value structure) ค่าในส่วนของฟิลด์ value อาจจะต้องผ่านการเข้ารหัสค่าตามโครงสร้าง TLV ด้วยเช่นกัน



รูปที่ 4-14 โครงสร้างของ TLV

4.8.2 ฟิลด์กำหนดชนิดข้อมูล (Type)

การกำหนดค่าให้กับฟิลด์ Type ขนาด 8 bit มีการจัดวางตำแหน่งบิต เพื่อให้ได้ค่าตัวเลขที่ใช้แทนกลุ่มชนิดข้อมูล แบ่ง เป็น 3 ส่วนย่อย ดังนี้



รูปที่ 4-15 รูปแบบการเข้ารหัสประเภทข้อมูล (type)

4.8.2.1 ฟิลด์ class 2 bits แรก : กำหนดประเภทข้อมูล มี 4 แบบ คือ

- 00 = ประเภท Universal เช่น ตัวเลขทั่วไป
- 01 = ประเภท Application ข้อมูลสำหรับใช้กับ application program
- 10 = ประเภท Context Specific ข้อมูลเจาะจง เช่น คำสั่งใน SNMP
- 11 = ประเภท Private ข้อมูลสำหรับใช้กรณีเฉพาะ

4.8.2.2 Form 1 bit ถัดมา : กำหนดว่าแบบข้อมูลนั้นเป็นแบบ พื้นฐาน(Primitive) หรือ แบบ โครงสร้าง(Constructed) เช่น ตัวอย่างแบบข้อมูลพื้นฐาน เช่น integer หรือ string ส่วนข้อมูลแบบโครงสร้าง ก็เช่น sequence

4.8.2.3 Tag number 5 bits สุดท้าย : เป็นส่วนกำหนดแบบข้อมูลซึ่งมีค่าได้ตั้งแต่ 0 ถึง 30

		แบบข้อมูล	value (ฐาน 16)
Universal	{	Integer	02
		OctectString	04
		null	05
		objectIdentifier	06
		sequence	16
Application - wide	{	IPAddress	40
		Counter	41
		Guage	42
		TimeTicks	43
Context - specific	{	get-request	A0
		get-next-request	A1
		get-response	A2
		set-request	A3
		trap	A4

รูปที่ 4-16 ตัวอย่าง type value กำหนดรูปแบบข้อมูลที่ใช้ใน SNMP

Example:

ข้อมูล Integer จะมี type เท่ากับ 02 หรือแยกออกมาเป็น bit ได้เป็น 0000 0010 ซึ่งมาจาก 00(Universal) , 0(Primitive) และ 0 0010(Tag Number)

4.8.3 ฟิลด์กำหนดความยาว (Length)

หากข้อมูลความยาวน้อยกว่า 128 bytes ฟิลด์นี้จะกินเนื้อที่เพียง 1 byte โดยมี bit ซ้ายสุดเป็น “0” และ 7 bit ที่เหลือกำหนดความยาว เช่น ข้อมูลยาว 10 byte ค่าในฟิลด์จะเท่ากับ 0x0A

ข้อมูลมีความยาวตั้งแต่ 128 bytes ขึ้นไป ต้องใช้ฟิลด์ length หลายออกเขต โดยที่บิตซ้ายสุดจะมีค่าเป็น “1” และใช้ 7 bits ที่เหลือเป็นตัวนับจำนวนออกเขตที่กำหนดความยาว ถัดจากนั้นจึงเป็นไปท์กำหนดความยาว เช่น ข้อมูลยาว 1000 bytes (03 E8) ค่าในฟิลด์จะเป็น 82 03 E8 โดยที่ 7 bits ขวาของ B2 คือ 000 0010 ซึ่งเท่ากับ 2 หมายถึงใช้ 2 bytes กำหนดความยาว และ 2 bytes นั้นคือ 03 E8

Value

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

 = 102

ข้อมูลความยาวน้อยกว่า 128 bytes

□ Short(0)/Long(1) form indicator

Value

1	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

0	1	1	1	0	0	1	1
---	---	---	---	---	---	---	---

0	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

= 7559605

□ Short/Long form indicator

■ Length of length

□ Length value

ข้อมูลความยาวมากกว่า 128 bytes

รูปที่ 4-17 รูปแบบการเข้ารหัสความยาว

4.8.4 ฟิลด์กำหนดข้อมูล (Value)

จะแตกต่างกันไปตามชนิดของข้อมูล

4.8.5 รูปแบบการเข้ารหัส TLV (เฉพาะบางประเภทข้อมูลที่ใช้ใน SNMP)

- ข้อมูลประเภทสายอักขระ ไม่ต้องเข้ารหัส ส่งไปตามลำดับของสายอักขระตามปกติ เช่น สายอักขระ “interfaces” จะอยู่ในรหัส TLV ดังนี้ 04 0A ‘i’ ‘n’ ‘t’ ‘e’ ‘r’ ‘f’ ‘a’ ‘c’ ‘e’ ‘s’
04 คือ OctetString และ 0A คือความยาว
- ข้อมูลตัวเลข
 - ค่าไม่เกิน 127 จะใช้เพียง 1 byte เท่านั้น
 - ค่าเกิน 127 จะต้องผ่านการเข้ารหัสอีกที คือ set bit ซ้ายสุดให้เป็น 1 และใช้ 7 bit ที่เหลือ กำหนดจำนวน octet ที่ต้องใช้ จากนั้นจึงค่อยตามด้วยค่า octet ถัดต่อไป เช่น ค่าตัวเลข 130 เมื่อเข้ารหัสแล้วจะได้ค่า 02 02 81 02 โดยที่ 02 คือ integer และ 81 02 แทนค่า 130
- Null ให้ส่งโดยเพียงแต่กำหนดค่าในฟิลด์ length เป็น 0 และไม่ต้องส่งค่าใดๆไปทั้งสิ้น
- ข้อมูลประเภท objectIdentifier ต้องเข้ารหัสด้วยวิธีพิเศษ

4.8.6 ตัวอย่าง SNMP Frame และการเข้ารหัส

การส่งคำสั่ง request (GetRequest PDU)

ลำดับ Byte เมื่อใช้คำสั่ง GetRequest PDU สอบถามค่า 1.3.6.1.2.1.1.1.0 หรือ sysDescr.0 แต่ละ

ฟิลด์ของ SNMP จะผ่านการเข้ารหัสตามโครงสร้าง TLV

UDP	30	27	02	01	00	04	06	73	23	6E
	6D	70	21	A0	1A	02	02	22	FB	02
	01	00	02	01	00	30	0E	30	0C	06
	08	2B	06	01	02	01	01	01	00	05
	00									

รูปที่ 4-18 SNMP Frame ของ GetRequest PDU

30	27									
02	01	00								
04	06	73	23	6E	6D	70	21			
A0	1A									
02	02	22	FB							
02	01	00								
02	01	00								
30	0E									
30	0C									
06	08	2B	06	01	02	01	01	01	00	
05	00									

Type	Length	value	หมายเหตุ
sequence	39	-	มีข้อมูล 39 bytes ตามมา
integer	1	0	version = 1
string	6	#snmp!	community = #snmp!
context-spc.	26	-	get request 26 bytes
integer	2	22FB	id = 22FB
integer	1	0	error status = 0
integer	1	0	error status = 0
sequence	14	-	
sequence	12	-	
oid	8	sysDescr.0	1.3.6.1.2.1.1.1.0
null	0	0	รหัสปิดท้าย

รูปที่ 4-19 ความหมายของการเข้ารหัสข้อมูลของ GetRequest PDU

การตอบกลับคำสั่ง request (GetResponse PDU)

ลำดับ byte เมื่อ agent ใช้คำสั่ง GetResponse PDU ตอบค่า sysDescr.0 ส่งกลับไป แต่ละฟิลด์ที่ผ่านการเข้ารหัสตามโครงสร้าง TLV จะแสดงได้ดังนี้ (เนื่องจากสายอักขระที่ตอบกลับมามีความยาวมากกว่า 300 byte ดังนั้นจึงเลือกแสดงแค่บางส่วนเท่านั้น)

UDP	30	81	F9	02	01	00	04	06	73	23
	6E	6D	70	21	A2	81	EB	02	02	02
	FB	02	01	00	02	01	00	30	81	DE
	30	81	DB	06	08	2B	06	01	02	01
	01	01	04	81	CE	43	65	73	63	6F

รูปที่ 4-20 SNMP Frame ของ GetResponse PDU

30	81	F9							
02	01	00							
04	06	73	23	6E	6D	70	21		
A2	81	EB							
02	02	22	FB						
02	01	00							
02	01	00							
30	81	DE							
30	81	DB							
06	08	2B	06	01	02	01	01	01	00
04	81	CE							
43	65	73	63	6F					

Type	Length	value	หมายเหตุ
sequence	377	-	มีข้อมูล 377 bytes ตามมา
integer	1	0	version = 1
string	6	#snmp!	community = #snmp!
context-spc.	363	-	get response 363 bytes
integer	2	22FB	id = 22FB
integer	1	0	error status = 0
integer	1	0	error index = 0
sequence	14	-	
sequence	12	-	
oid	8	sysDescr.0	1.3.6.1.2.1.1.1.0
OctectString	334	-	
			Cisco.....

รูปที่ 4-21 ความหมายของการเข้ารหัสข้อมูลของ GetResponse PDU